

Running the Earnings-Call Automation on the Mac Minis

A practical assessment and step-by-step plan

The short version

It can be done, and most of the system moves across untouched. But this is a port, not a copy. Roughly 90% of the code is plain Node.js that runs anywhere; the remaining 10% is the part that makes recordings actually start, and it is written directly against the Windows API.

There is also one question that has to be answered before any of the work is worth starting, and it can be tested in about ten minutes on a single machine. It is covered in Step 1.

Part 1 - What actually has to change

1.1 The keystroke injector (the hard part)

Every recording depends on one thing: Chrome must be given a real, operating-system-level keypress while the right tab is in front. This is not a convenience. Chrome only grants the extension permission to capture a tab in response to a genuine user gesture, so a simulated or scripted "click" is refused. That is why the current design goes to such lengths here.

On Windows this is `src/send-shortcut.ps1` - 262 lines of PowerShell calling directly into the Win32 API: `SetForegroundWindow`, `AttachThreadInput`, `SendInput`, `EnumWindows`. None of that exists on macOS. The file has to be replaced entirely.

The good news: the macOS equivalent is far simpler. Where Windows actively refuses to let a background process take the foreground - which is why that script has a retry loop and an ALT-tap workaround - macOS just does it:

```
tell application "Google Chrome" to activate
tell application "System Events"
    keystroke "y" using {command down, shift down}
end tell
```

That is the whole thing. Perhaps twenty lines with error handling, replacing 262.

1.2 The Chrome flag check

`src/preflight.js` inspects the running Chrome process to confirm it was launched with the flag that lets captures start without a consent prompt. It reads the Windows process table. On macOS the same information comes from `ps -ax -o command`. Small change, maybe fifteen lines.

1.3 The extension shortcut

The extension currently defines only a Windows shortcut. It needs a Mac one added, because Ctrl is not the modifier macOS uses:

```
"commands": {  
  "trigger-transcription-popup": {  
    "suggested_key": {  
      "default": "Ctrl+Shift+Y",  
      "mac": "Command+Shift+Y"  
    },  
    "description": "Open the transcription popup"  
  },  
}
```

Worth checking that Chrome on macOS does not already use Command+Shift+Y for something. If it does, pick another combination - the exact keys do not matter, they only have to match what the config tells the injector to send.

1.4 Nothing else in the codebase is Windows-specific

This is genuinely good news and it is worth stating plainly. I checked every source file: the only two that touch Windows are the two above. Everything else - reading the portal table, working out which calls are due, resolving webcast links, filling registration forms, walking Zoom and Teams join screens, the retry logic, the reporting - is ordinary cross-platform Node.js.

Part 2 - The constraint that decides everything

2.1 These machines cannot be treated as headless servers

This is the most important point in the document.

Because the recording depends on a real keypress landing on a real window, each Mac mini needs a genuine graphical login session, awake, unlocked, with Chrome visible. A plain SSH connection has no desktop attached to it, so nothing you start over SSH alone will ever be able to record.

What this means in practice, per machine:

- Automatic login to a desktop session at boot - not a login screen waiting for a password
- Sleep disabled, and the display prevented from sleeping
- Screen lock and screen saver switched off
- Chrome launched inside that desktop session, not from an SSH shell

Apple's own Screen Sharing helps here: unlike Windows Remote Desktop, disconnecting from a Mac over Screen Sharing leaves the desktop session running rather than locking it. So you can connect, check on things, and disconnect without stopping the work. That is a real advantage of Macs for this particular job.

2.2 macOS will ask for two permissions, once per machine

macOS blocks synthetic keystrokes and screen access by default. Both must be granted by hand the first time,

in System Settings > Privacy & Security:

- **Accessibility** - for whatever process sends the keystroke (Terminal, or the tool itself). Without it, the keystroke is silently swallowed and every recording fails to start.
- **Screen Recording** - Chrome needs this to capture tab audio.

These cannot be granted over SSH or by script; they need a few clicks on each machine. Budget for it, and do it once during setup.

Part 3 - What are the seven machines for?

This needs deciding before anything is installed, because it changes the work substantially.

Option A - Capacity: split the calls seven ways

Each machine handles a seventh of the calls. Seven times the throughput, and no machine is doing much work at any moment.

The catch: right now every watcher reads the same table and decides independently what to record, keeping its own local record of what it has claimed. Two machines running as-is would both record the same call - duplicate transcripts, and two browsers fighting for the same webcast.

The fix is straightforward and needs no shared database. Give each machine a number, and have it ignore any call that is not "its own":

```
// config.local.json on machine 3 of 7
{ "shardIndex": 2, "shardCount": 7 }

// each machine keeps only the rows that hash to its own number
if (hashOf(symbol) % shardCount !== shardIndex) continue;
```

Because the calculation depends only on the symbol, all seven machines reach the same conclusion without ever talking to each other. Machine 3 knows AAPL is not its job, and it knows that without asking anybody. Perhaps thirty lines of code and a test.

Option B - Redundancy: all seven watch everything

More resilient to one machine dying, but they would all record every call seven times over. It needs a shared lock - a small database or file share that every machine checks before claiming a call. More moving parts, more that can fail, and a genuinely harder thing to get right.

Recommendation

Start with Option A. It is simpler, needs no shared infrastructure, and the failure mode is mild: if one machine goes down you lose a seventh of that window's calls, and the reconciliation report on that machine tells you exactly which ones. You can always add redundancy later; you cannot easily remove complexity once it is in.

Part 4 - Step by step

Step 1 - Prove the keystroke works (do this first, on ONE machine)

Do not skip this and do not do it last. Everything else depends on it, and it is the one thing I cannot verify from here.

The question: when macOS sends a synthetic Command+Shift+Y, does Chrome treat it as a real user gesture and grant the extension permission to capture the tab?

This matters because the equivalent question had a surprising answer on Windows. There, the obvious method - PowerShell's SendKeys - put the right window in the foreground, verifiably so, and yet the extension shortcut never fired at all: the popup simply did not open. SendKeys delivers keystrokes by posting window messages, and Chrome's command layer does not accept that as a genuine keypress. Only SendInput, which injects at the same level as real hardware, worked.

So the thing to watch for is whether the popup opens. If it does, macOS's synthetic keystroke has cleared the same hurdle SendInput had to clear, and the rest should follow - though it is still worth confirming a real transcript comes back, since the permission that lets the extension capture the tab is granted by a separate mechanism from the one that opens the popup.

The test, roughly ten minutes:

1. Install Chrome and load the extension on one Mac mini.
2. Add the Mac shortcut to the extension manifest (section 1.3) and reload it.
3. Grant Accessibility and Screen Recording permissions.
4. Open any page with audio - a YouTube video is ideal.
5. From Terminal, run the AppleScript from section 1.1.
6. Confirm the popup opens, then start a transcription and check that real text comes back.

If the popup opens, the main risk is behind us - that is the exact hurdle SendKeys failed on Windows. If step 6 then produces a real transcript, the port is straightforward and everything below applies.

If the popup does not open, stop. It would mean macOS treats an AppleScript keystroke the way Windows treats SendKeys, and we need a different injection method - a small native helper, or a hardware keyboard emulator. Both are a different and much larger piece of work, and worth deciding on together rather than pressing ahead.

Step 2 - Prepare each machine

1. **Node.js 22 or newer.** The tool checks this at startup and refuses to run on anything older, because the extension popup is driven over a WebSocket that older versions do not provide.
2. **Google Chrome**, plus the extension loaded unpacked.
3. **Auto-login, no sleep, no screen lock** (section 2.1).
4. **Accessibility and Screen Recording permissions** granted (section 2.2).
5. **Log in to the admin portal in that Chrome profile** and leave it logged in.

Step 3 - Install the extension at the SAME path on every machine

A detail that will save real annoyance. Chrome derives an unpacked extension's ID from the folder it lives in, so the same folder path on every machine produces the same ID - and then one configuration file works everywhere. Different paths mean seven different IDs and seven different config files to keep straight.

Put it at exactly the same absolute path on all seven, for example `/Users/fmp/extension`. The tidier long-term answer is to package the extension properly so its ID is fixed regardless of location, which is worth doing eventually.

Step 4 - Deploy the code

```
git clone <repo> ~/call-watcher
cd ~/call-watcher/call-watcher
npm install
npm run test:unit          # ~1 second, no browser needed
```

If those tests pass, the machine-independent half of the system is working on that Mac. It is a genuinely useful checkpoint: it confirms Node, the code and the configuration are all sound before anything touches Chrome.

Step 5 - Configure each machine individually

`config.local.json` is deliberately not stored in git, precisely so each machine can differ:

```
{
  "extensionId": "...",      // same everywhere if Step 3 was followed
  "shardIndex": 2,          // 0 to 6 - unique per machine
  "shardCount": 7
}
```

Step 6 - Start Chrome with the required flags

```
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
--remote-debugging-port=9222 \
--user-data-dir="$HOME/ChromeDebugProfile" \
--auto-accept-this-tab-capture &
```

The last flag is the one people forget, and its absence is silent: everything appears to work, right up to the point where a system dialog no automation can dismiss blocks each capture. The tool checks for the flag at startup and complains loudly if it is missing.

Step 7 - Start the watcher, and keep it started

```
npm run supervise
```

This restarts the watcher if it exits, and also if it is running but has gone blind - a stale heartbeat, a dropped Chrome connection, an expired portal session.

To have it start by itself at login, use a macOS Launch Agent (not a Launch Daemon - daemons run without a desktop, which is exactly what will not work here). A short plist in `~/Library/LaunchAgents/` is enough. Pair it with `caffeinate` so the machine never sleeps mid-call.

Step 8 - Verify each machine before trusting it

- `npm run test:unit` - the logic, in about a second

- `npm test` - the full suite including browser-driven tests, a few minutes
 - `npm run report` - at the end of the day, on each machine. The number to look at is "unaccounted": it should always be zero. Anything else means a call reached its window and left no trace, which is the one failure worth chasing.
-

Part 5 - Effort, honestly

Assuming the Step 1 test succeeds:

- **Replace the keystroke injector** - half a day, most of it verifying rather than writing
- **Make the Chrome flag check cross-platform** - an hour
- **Add the shard filter** - half a day with tests
- **Set up the first machine end to end** - half a day, mostly permissions and Chrome
- **Each additional machine** - an hour or so once the first is proven

Call it two to three days for a working seven-machine setup, plus a day of running it alongside the current Windows machine before relying on it.

If the Step 1 test fails, stop and tell me. The alternatives - a small native helper, or driving a hardware keyboard emulator - are a different and much larger piece of work, and we should decide together whether it is worth it.

Part 6 - Things that will bite if nobody plans for them

- **A locked screen stops everything.** No desktop, no keystroke, no recording. This is the single most likely cause of a quiet, total failure.
 - **A macOS update can reset privacy permissions** and re-lock the machines. Worth checking after any update, and worth knowing when something stops working the morning after one.
 - **Seven machines mean seven sets of reports.** Each one answers "did I get my share?" but nothing yet answers "did we get everything, across all seven?". Straightforward to add - the reports are plain files - but it does not exist today.
 - **Each machine needs its own portal login session** and each will expire independently.
 - **The Windows machine and the Macs must not both record the same calls.** If it stays running, give it a shard number too.
-

What I would do next

1. Run the Step 1 keystroke test on one Mac mini. Nothing else is worth starting until that question is answered, and it takes about ten minutes.
2. Tell me the result. If it works, I will write the Mac injector, make the flag check cross-platform, and add

the shard filter - with tests, as with everything else here.

- 3.** Set up one machine completely and run it for a day beside the existing one, comparing the reports from both.
- 4.** Roll out to the remaining six, which should be mostly repetition at that point.